

UCS503 Software Engineering Project Report

LEXIA

An AI Powered Accessibility Reading Platform for People with Dyslexia

Submitted by:

1024170445 Nandini Sharma, CSED
1024170435 Divyadeep Singh, CSED

BE Third Year, Computer Science and Engineering

Submitted to:

Mrs. Nisha Thakur

Project Report

**Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala**

Abstract

LEXIA is a web-based accessibility reading platform designed to reduce reading fatigue and comprehension barriers for people with dyslexia and related reading difficulties. The system provides a calm, customizable reading space where users can adjust font, text size, letter spacing, line spacing, page tint, and focus mode. It also provides multi-level AI text simplification, contextual vocabulary support, browser-based read-aloud assistance, and a personal article library.

The project uses a React and Tailwind CSS frontend, a small Django service for secure AI orchestration, Google Gemini for language transformations, and Supabase for authentication, database storage, and file storage. The report presents the problem context, requirements, design, database model, working flow, implementation plan, testing criteria, and twelve-week schedule. The intended outcome is a practical assistive reading system that is responsive, safe to use, and adaptable to individual reading preferences.

Acknowledgement

We express our sincere gratitude to Mrs. Nisha Thakur for her guidance and support throughout the planning and development of this project. We also thank the Department of Computer Science and Engineering, Thapar Institute of Engineering and Technology, for providing the academic environment and resources needed to undertake this work.

Contents

Abstract	i
Acknowledgement	ii
1 Introduction	2
1.1 Background	2
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope	3
2 Requirements Analysis	4
2.1 Stakeholders	4
2.2 Functional Requirements	4
2.3 Non Functional Requirements	5
3 System Design	6
3.1 Architecture	6
3.2 UML Design Diagrams	6
3.3 Simplify Text Flow	12
3.4 Assessment Flow	13
4 Database Design and Security	14
4.1 Data Model	14
4.2 Access Control	14
5 Implementation Plan	15
5.1 Development Method	15
5.2 Current Implementation Status	15
5.3 Planned Increments	15
6 Testing and Evaluation	16
6.1 Testing Strategy	16
6.2 Evaluation Criteria	16
7 Project Schedule	17
7.1 Twelve Week Gantt Chart	17
8 Risks and Mitigation	18
9 Conclusion	19

Introduction

1.1 Background

Digital reading is central to academic and professional life, yet conventional web pages often create unnecessary barriers for readers with dyslexia. Dense paragraphs, rigid typography, visual crowding, difficult vocabulary, and limited reading controls can increase cognitive load and reduce comprehension. Most existing reading tools solve only one part of the problem, such as text-to-speech or font enlargement, without offering a unified and personalized workflow.

LEXIA addresses this gap through an accessible reading environment. It allows a user to enter text, adapt its presentation, request a simpler version, listen to it, and save it for later use. The platform is designed as an assistive tool, not as a clinical diagnostic system.

1.2 Problem Statement

Individuals with dyslexia and related reading challenges may experience eye-tracking fatigue, letter confusion, difficulty processing long sentences, and vocabulary overload while reading digital content. Standard interfaces generally do not allow the reader to control typography, visual contrast, or reading pace in a consistent way. A web-based system is needed that combines adaptive formatting with AI-assisted language support while preserving the intended meaning of source material.

1.3 Objectives

The objectives of LEXIA are:

- to provide a dyslexia-friendly reading interface with adjustable visual settings;
- to simplify text at Basic, Easier, and Very Simple levels using controlled AI prompts;
- to provide read-aloud assistance through the Web Speech API;
- to support saved articles, reading history, and persistent user preferences;
- to provide assessments, practice exercises, progress tracking, and supervised views for parents or teachers in subsequent modules; and
- to maintain secure authentication and role-based data access.

1.4 Scope

The current scope includes user registration and login, adaptive reading controls, text simplification, text-to-speech, article saving, and the database foundation for assessment and progress features. Assessment analysis, recommendations, practice content, parent/teacher linking, and reporting are planned as the next implementation increments. LEXIA does not diagnose dyslexia and does not replace professional educational or clinical support.

Requirements Analysis

2.1 Stakeholders

Stakeholder	Need from LEXIA
Student	A readable, low-distraction environment with controls that support individual reading needs.
Parent or Guardian	A safe view of linked student progress and recommendations.
Teacher	A structured view of linked learners' assessment progress and areas needing practice.
Project Team	A modular and secure architecture that can be tested and extended incrementally.

Table 2.1: Primary stakeholders and their needs

2.2 Functional Requirements

ID	Requirement	Description
FR1	Authentication and roles	The system shall support registration and login for Student, Parent/Guardian, and Teacher roles.
FR2	Reading preferences	The user shall be able to save font, size, letter spacing, line spacing, page tint, and focus-mode preferences.
FR3	Text input	The user shall be able to paste text and later upload supported documents.
FR4	AI simplification	The user shall be able to request Basic, Easier, or Very Simple text simplification.
FR5	Vocabulary help	The user shall be able to request a contextual explanation for a selected difficult word.
FR6	Read aloud	The user shall be able to start and stop browser-based speech synthesis for displayed text.

FR7	Library and history	The user shall be able to save articles and view reading history.
FR8	Assessment	The system shall support word recognition, spelling, reading passage, and optional voice-reading assessments.
FR9	Progress reports	The system shall store scores, error patterns, recommendations, milestones, and report history.
FR10	Linked access	A parent or teacher shall view only the progress of students linked through an approved relationship.

Table 2.2: Functional requirements

2.3 Non Functional Requirements

- **Accessibility:** keyboard-operable controls, readable typography, clear focus states, and a calm visual hierarchy.
- **Performance:** target median end-to-end simplification latency of at most three seconds for a standard article of up to 800 words.
- **Security:** browser code uses only the Supabase publishable key; Gemini and service-role credentials remain server-side.
- **Reliability:** errors from the AI service must show a retry-friendly message without exposing internal credentials or provider details.
- **Scalability:** the frontend, AI service, and hosted database are independently deployable.

System Design

3.1 Architecture

LEXIA follows a lightweight client-service architecture. The React frontend manages user interaction, reading settings, and browser-native speech synthesis. Supabase provides authentication, database tables, Row Level Security policies, and private object storage. A separate Django service verifies the Supabase access token before it calls Gemini. This boundary prevents the Gemini key and Supabase server credentials from reaching the browser.

Layer	Responsibility
React and Tailwind CSS	Authentication screens, reading interface, accessibility controls, article library views, and assessment screens.
Django AI service	Validates authenticated requests, enforces input limits, forms safe prompts, and calls Gemini.
Google Gemini API	Returns simplified text, summaries, contextual explanations, recommendations, and structured analysis when enabled.
Supabase	Hosts authentication, PostgreSQL data, private file storage, and Row Level Security policies.
Web Speech API	Provides browser-native text-to-speech without storing audio recordings by default.

Table 3.1: LEXIA architecture components

3.2 UML Design Diagrams

The following UML diagrams document the principal user interactions and the internal flow of LEXIA. The diagrams are included as part of the system design and are consistent with the documented Simplify Text workflow and system architecture.

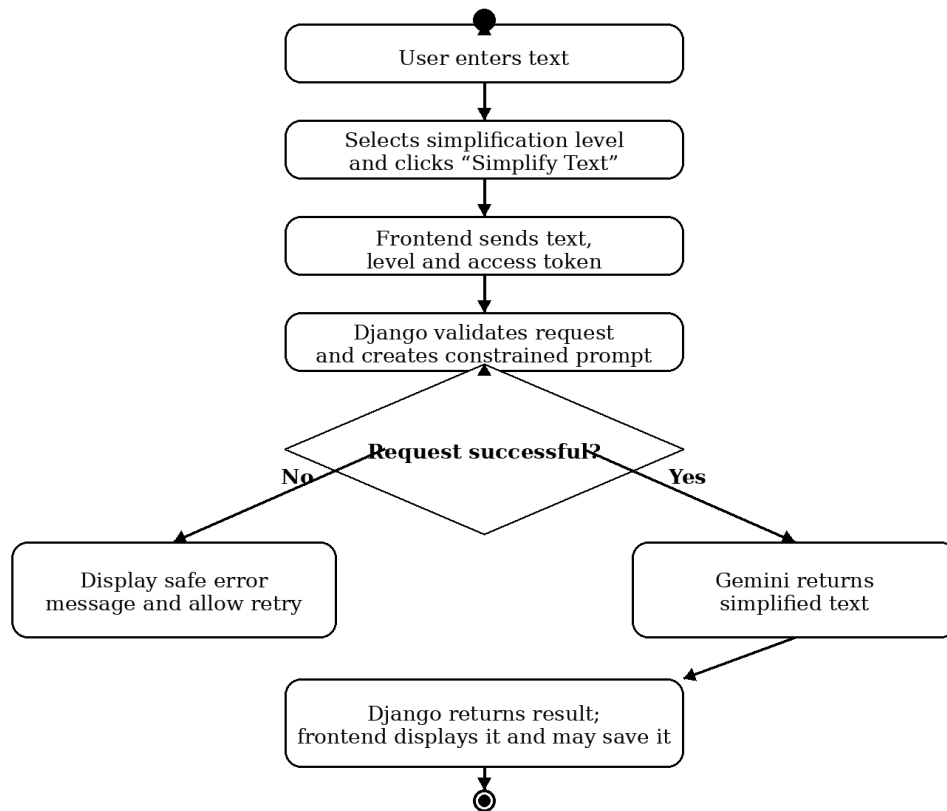
Activity Diagram - LEXIA (Simplify Text)

Figure 3.1: Activity diagram for the Simplify Text workflow

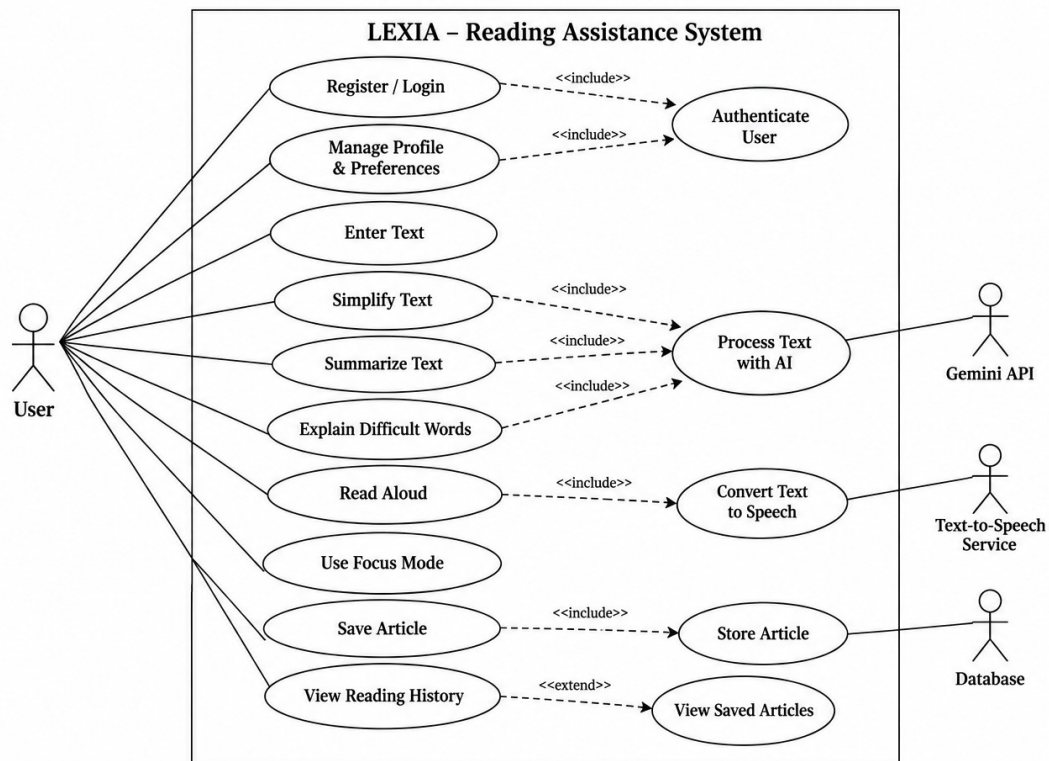


Figure 3.2: Use case diagram of the LEXIA reading assistance system

LEXIA – Use Case Templates

UC-01: Register / Login		UC-02: Manage Profile & Preferences	
Use Case ID	UC-01	Use Case ID	UC-02
Use Case Name	Register / Login	Use Case Name	Manage Profile & Preferences
Actor(s)	User	Actor(s)	User
Description	Allows a new user to create an account or an existing user to log in to LEXIA.	Description	Allows the user to view and update their profile information and reading preferences.
Preconditions	User is on the LEXIA website.	Preconditions	User is logged in.
Main Flow	1. User selects Sign Up or Login. 2. User enters required details (name, email, password). 3. System validates the credentials. 4. User is successfully registered/logged in.	Main Flow	1. User opens profile/settings. 2. User updates preferences (font, size, spacing, theme, etc.). 3. System saves the changes. 4. Updated settings are applied to the user interface.
Postconditions	User gains access to their account.	Postconditions	User's preferences are stored and reflected in the system.
Alternate Flow	A1: Invalid credentials → system shows error message. A2: Email already exists → system prompts user to login.	Alternate Flow	A1: Invalid input → system shows error message. A2: User cancels changes → no updates are saved.

UC-03: Enter Text		UC-04: Simplify Text	
Use Case ID	UC-03	Use Case ID	UC-04
Use Case Name	Enter Text	Use Case Name	Simplify Text
Actor(s)	User	Actor(s)	User, Gemini API
Description	Allows the user to input or paste the text they want to read and understand.	Description	Allows the user to get a simplified version of the input text using AI.
Preconditions	User is logged in.	Preconditions	User has entered text.
Main Flow	1. User navigates to the reading page. 2. User pastes or types text into the input area. 3. User clicks 'Analyze' or selects a feature. 4. System accepts the text and prepares it for processing.	Main Flow	1. User clicks 'Simplify'. 2. System sends the text to Gemini API. 3. Gemini API returns a simplified version. 4. System displays the simplified text to the user.
Postconditions	Text is successfully submitted for processing.	Postconditions	Simplified text is shown to the user.
Alternate Flow	A1: Text is empty → system prompts user to enter text. A2: Text exceeds limit → system shows error message.	Alternate Flow	A1: API error → system shows error message. A2: No response from API → user can retry.

UC-05: Summarize Text		UC-06: Explain Difficult Words		UC-07: Read Aloud	
Use Case ID	UC-05	Use Case ID	UC-06	Use Case ID	UC-07
Use Case Name	Summarize Text	Use Case Name	Explain Difficult Words	Use Case Name	Read Aloud
Actor(s)	User, Gemini API	Actor(s)	User, Gemini API	Actor(s)	User
Description	Allows the user to get a summary of the input text using AI.	Description	Allows the user to get simple explanations of difficult words with examples.	Description	Allows the user to listen to the text using text-to-speech.
Preconditions	User has entered text.	Preconditions	User has text displayed.	Preconditions	Text is available on the reading page.
Main Flow	1. User clicks 'Summarize'. 2. System sends the text to Gemini API. 3. Gemini API returns a summary. 4. System displays the summary to the user.	Main Flow	1. User selects a difficult word. 2. System sends the word and context to Gemini API. 3. Gemini API returns an explanation (with example). 4. System displays the explanation to the user.	Main Flow	1. User clicks 'Read Aloud'. 2. System converts the text to speech using Web Speech API. 3. Audio is played with controls (pause, resume, speed). 4. User can stop or adjust the playback.
Postconditions	Summary is shown to the user.	Postconditions	Word explanation is shown to the user.	Postconditions	Text is read aloud to the user.
Alternate Flow	A1: API error → system shows error message. A2: Empty or unclear text → system prompts for valid text.	Alternate Flow	A1: No word selected → system prompts user to select a word. A2: API error → system shows error message.	Alternate Flow	A1: Browser does not support speech → system shows error message. A2: User stops playback → audio is paused.

Figure 3.3: Use-case templates for major LEXIA functions

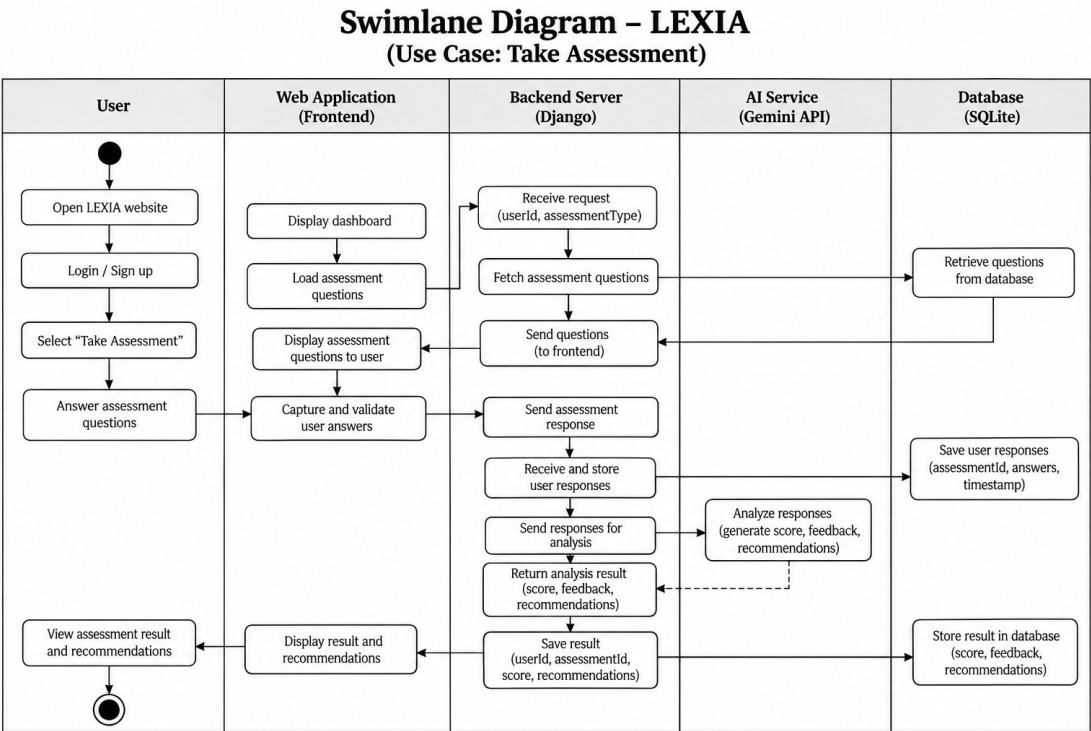


Figure 3.4: Swimlane diagram for the Take Assessment use case

Sequence Diagram – LEXIA (Simplify Text)

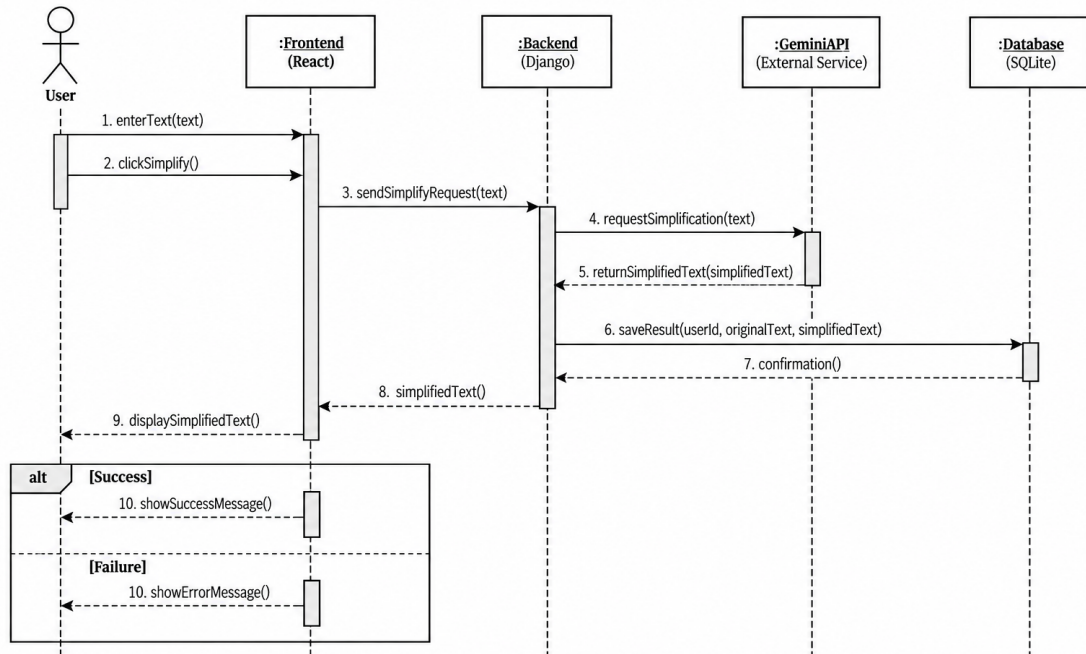


Figure 3.5: Sequence diagram for the Simplify Text use case

Collaboration Diagram (Simplify Text) – LEXIA

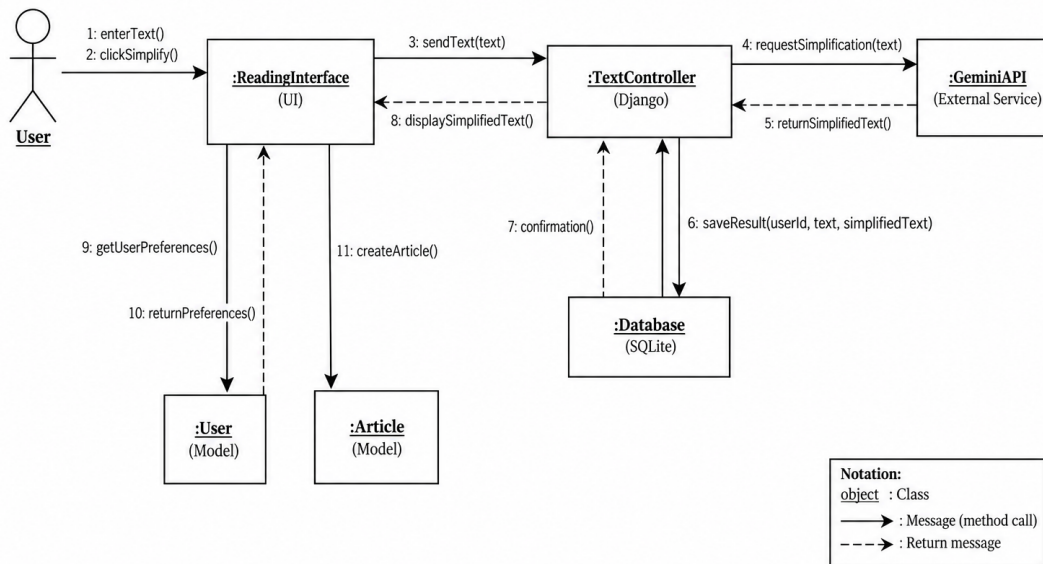


Figure 3.6: Collaboration diagram for the Simplify Text use case

3.3 Simplify Text Flow

The Simplify Text use case operates as follows:

1. The authenticated user pastes or enters text in the reading interface.
2. The user selects a simplification level and presses *Simplify Text*.
3. The React client sends the text, selected level, and Supabase access token to the Django endpoint.
4. Django validates the token through Supabase JWKS, verifies the text length, and creates a constrained Gemini prompt.
5. Gemini returns a simplified version that preserves the original meaning.
6. Django returns the result to the frontend. The frontend displays it and may save it in the personal library.

If the request fails, the application displays a safe error message and gives the user the option to retry. The client never sends Gemini credentials directly to the browser.

3.4 Assessment Flow

The assessment module is designed around Word Recognition, Spelling, Reading Passage, and Voice-Based Reading activities. A session records the type of assessment, responses, duration, and submission status. Local scoring provides immediate checks where possible. A structured summary may be sent to Gemini through Django to identify error patterns and produce personalized practice recommendations. The saved report then becomes available to the student and, where permitted, linked parent or teacher accounts.

Database Design and Security

4.1 Data Model

The Supabase PostgreSQL schema uses the tables below.

Table	Purpose
profiles	Stores the authenticated user identity, role, profile information, and reading preferences.
articles	Stores user-owned original text, simplified versions, source type, and optional storage path.
reading_history	Tracks article reading sessions and completion progress.
assessments	Stores each assessment session, type, timestamps, and status.
answers	Stores question-level responses and correctness information.
scores	Stores the overall score and skill-level breakdown for an assessment.
error_patterns	Stores identified patterns such as letter reversal or vowel-blending errors.
recommendations	Stores personalized, prioritized practice suggestions.
progress_history	Stores time-series values for charts and milestones.
student_links	Stores approved parent/guardian or teacher relationships with a student.

Table 4.1: Core data entities

4.2 Access Control

Row Level Security is applied to all user data. Students can read and modify their own profiles, articles, reading history, and assessment data. Parent and teacher roles can read data only for students with whom they have an approved link. Article files are stored in a private bucket with file paths scoped to the authenticated owner. Creation of a student link is intentionally handled through an approval or trusted server flow so that an observer cannot attach an arbitrary student account.

Implementation Plan

5.1 Development Method

The project follows incremental development. Each increment is independently demonstrable, beginning with the reader flow that provides immediate user value. This approach allows accessibility feedback to influence subsequent work on assessment and progress reporting.

5.2 Current Implementation Status

The first working increment includes a React reading interface that preserves the project visual prototype, Supabase authentication calls, persistent reading preferences, browser text-to-speech, article saving, a secure Django health endpoint, and an authenticated text simplification endpoint. The database migration defines the complete foundation for user, article, assessment, score, recommendation, progress, and relationship data.

5.3 Planned Increments

1. Complete the personal library, reading history, upload support, and contextual difficult-word explanation.
2. Implement the assessment session, local scoring, results screen, and AI-assisted feedback.
3. Implement practice exercises, progress charts, milestones, report history, and linked parent/teacher dashboards.
4. Add automated tests, accessibility checks, error monitoring, deployment configuration, and pilot usability evaluation.

Testing and Evaluation

6.1 Testing Strategy

Testing will combine unit, integration, accessibility, and user-oriented evaluation. Frontend tests will check interface state and accessibility controls. Django tests will validate authentication, request validation, prompt construction, and error handling. Integration tests will verify the complete path from the reader screen to Django, Gemini, and the persisted article record. Row Level Security tests will verify that users cannot access another user’s data or unapproved student records.

6.2 Evaluation Criteria

Metric	Target or Method
Simplification latency	Median end-to-end latency no greater than 3.0 seconds for an article of up to 800 words.
Readability improvement	Compare Flesch-Kincaid Grade Level before and after simplification.
Comprehension and speed	Compare task completion time and short comprehension questions on original and adapted text.
Usability	Conduct a pilot study with 10–15 student volunteers and target a System Usability Scale score of at least 80.
Accessibility	Check keyboard navigation, clear focus indication, readable settings, and contrast against WCAG 2.1 AA expectations.

Table 6.1: Evaluation criteria

Project Schedule

7.1 Twelve Week Gantt Chart

The Gantt chart below shows the planned weeks for each activity. A check mark indicates an active week.

Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Requirement Analysis	✓	✓										
User Research and Personas	✓	✓										
UI UX Design and Wireframes		✓	✓									
Database Design		✓	✓									
Backend Development			✓	✓	✓							
Frontend Development			✓	✓								
Gemini API Integration					✓	✓	✓					
Text to Speech						✓	✓					
Accessibility and Focus Mode						✓	✓	✓				
Frontend Backend Integration							✓	✓				
Testing and Bug Fixing									✓	✓		
Usability Testing									✓	✓		
Deployment										✓		
Documentation											✓	✓
Final Presentation											✓	✓

Table 7.1: LEXIA development schedule

Risks and Mitigation

Risk	Potential effect	Mitigation
AI output distortion	Simplified text could change intended meaning.	Use conservative prompts, show original text, and retain user control.
API latency or quota limits	Delayed response or failed transformation.	Enforce input limits, show loading feedback, use retries, and cache suitable results.
Speech compatibility	Browser speech voices and controls vary.	Use Web Speech API capability checks and provide clear stop and retry controls.
Unauthorized data access	Personal reading or assessment data may be exposed.	Apply Supabase Row Level Security and server-side credential isolation.
Accessibility gaps	Controls may not work for all users.	Conduct keyboard, contrast, and usability checks throughout development.

Table 8.1: Project risks and planned mitigation

Conclusion

LEXIA proposes a practical, web-based response to common digital reading barriers. By combining adaptable typography, visual reading aids, AI-assisted simplification, text-to-speech, and secure personal data storage, the platform can support different reading preferences without requiring a separate desktop application. The staged development plan prioritizes a usable reader first and then extends it through assessment, practice, progress analysis, and supervised views. The project will be evaluated through performance, readability, accessibility, and pilot usability measures before final deployment.

Bibliography

- [1] Meta Open Source. *React Documentation*. <https://react.dev/>.
- [2] Django Software Foundation. *Django Documentation*. <https://docs.djangoproject.com/>.
- [3] Supabase. *Supabase Documentation*. <https://supabase.com/docs>.
- [4] Google AI for Developers. *Gemini API Documentation*. <https://ai.google.dev/gemini-api/docs>.
- [5] MDN Web Docs. *Web Speech API*. https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API.